

2025

API REST

Projet en groupe



Jessica Grisales, Francisco

Sousa, Diogo Lopes

FID2

10/10/2025

Table des matières

Introduction	2
Analyse	3
Maquette.....	7
Réalisation de l'API REST.....	9
Création des routes	13
Test des routes	14
Erreurs rencontrés	15
Conclusion.....	16
Webographie / Bibliographie / Glossaire	16
Annexes	16
Usage de l'IA.....	17

Introduction

1.1 Contexte, Problématique et Objectifs

Le projet "**API REST**" avec **Adonis** consistait à concevoir et implémenter une **API RESTful** robuste, performante. L'objectif principal était de garantir une **exploitation et une livraison professionnelle** du backend, en démontrant la maîtrise des concepts d'architecture, de sécurité, de gestion des données et de validation. Le livrable final doit non seulement fonctionner, mais aussi être **documenté** (Swagger) et **testé** (Bruno), assurant son intégration aisée avec un frontend.

1.2 Stack Technologique

Le projet s'est conformé au stack technologique de référence du module :

- **Plateforme : Node.js** et **npm** pour l'exécution et la gestion des dépendances.
- **Framework d'Application : AdonisJS**, reconnu pour sa structure opinionnée et son accent sur la sécurité et la productivité, offrant une expérience du pattern **MVC** (Modèle-Vue-Contrôleur).
- **Système de Gestion de Base de Données (SGBD) : MySQL**.
- **ORM : Lucid**, l'ORM natif d'AdonisJS, pour l'abstraction et l'interaction sécurisée avec la base de données.
- **Outils : Swagger** pour la documentation de l'API et **Bruno** pour les tests d'intégration manuels.

Analyse

2.1 Planification et Organisation des Tâches

Bien qu'un outil formel de suivi (GitHub Project) soit requis, l'organisation de l'équipe (Jessica, Diogo, Francisco) a permis une gestion modulaire et efficace. Le travail a été décomposé par **ressource métier** :

- **Jessica** : Responsable du cycle de vie complet (Book).
- **Diogo** : Responsable du cycle de vie complet (Writer).
- **Francisco** : Responsable du cycle de vie complet (Category et User).

Cette approche a minimisé les conflits de fusion sur le code et a permis un développement parallèle rapide sur des branches différentes avant la fusion des branches dans le main. Les étapes de développement ont systématiquement inclus : migration, modèle Lucid, validateur VineJS, contrôleur CRUD, avant de faire le routage à trois pour faciliter les connections entre les différentes parties.

2.2 Analyse de l'API REST

L'API est conçue autour du paradigme **RESTful**, utilisant les verbes HTTP (GET, POST, PUT/PATCH, DELETE) pour manipuler les ressources. Le choix de la méthode `router.resource().apiOnly()` dans AdonisJS garantit la génération automatique des sept routes standards d'une API CRUD pour chaque ressource, assurant la cohérence et le respect des conventions.

2.2.1 Détail du Routage et de l'Architecture des Ressources

L'analyse du fichier `routes.ts` révèle une architecture de routage standard et une structure de routage **imbriquée (nested routing)** :

1. Routage Principal (books) :

```
router.resource('books', BooksController).apiOnly()
```

Cette ligne expose directement l'entité centrale (`/books`), permettant un accès de haut niveau pour la lecture, la création, la modification et la suppression.

2. Routage Imbriqué et Contextuel :

```
router.get('category/ :category_id/books', [BooksController, 'indexByCategory'])
```

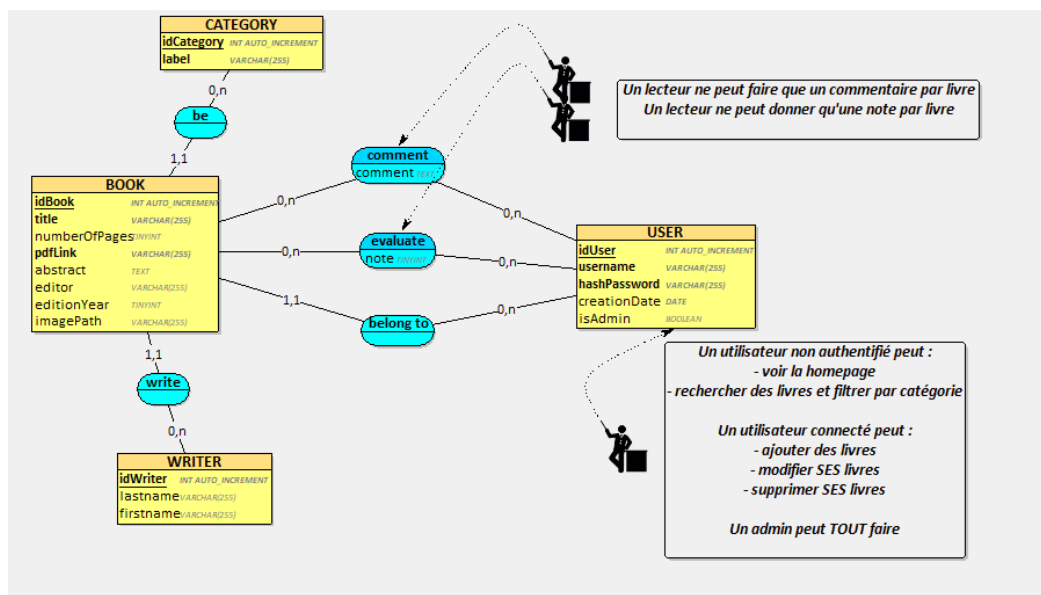
cette route introduit une relation hiérarchique claire entre les **catégories** et les **livres**. Ce schéma implique qu'une requête adressée à GET /category/2/books est destinée à récupérer tous les livres appartenant à la catégorie ayant l'ID 2.

- **Implication Métier** : Ce choix de routage souligne la dépendance fonctionnelle des livres vis-à-vis de leur catégorie. Le contrôleur BooksController doit exploiter le paramètre dynamique category_id pour filtrer les livres en conséquence et ne retourner que ceux qui appartiennent à cette catégorie précise. Dans ce contexte, cette route permet d'obtenir la liste des livres associés à chaque catégorie, en respectant la logique métier et la structure relationnelle du domaine.

2.3 Modèle Conceptuel et Logique de Données (MCD / MLD / MPD)

MCD (Modèle Conceptuel de Données)

Modèle fournit par l'enseignant, il est la pièce centrale de référence du projet. Nous nous sommes appuyés sur lui pour mettre en place la structure et la répartition du travail. De plus que les bases de la structure de la base de données sera directement élaborée à partir de ce modèle.



AUTEUR : JESSICA GRISALES, FRANCISCO SOUSA, DIOGO LOPES

CRÉÉ LE MERCREDI, 14 MAI 2025

MODIFIÉ PAR : JESSICA GRISALESB

IMPRIMÉ PAR JESSICA GRISALES, FRANCISCO SOUSA,

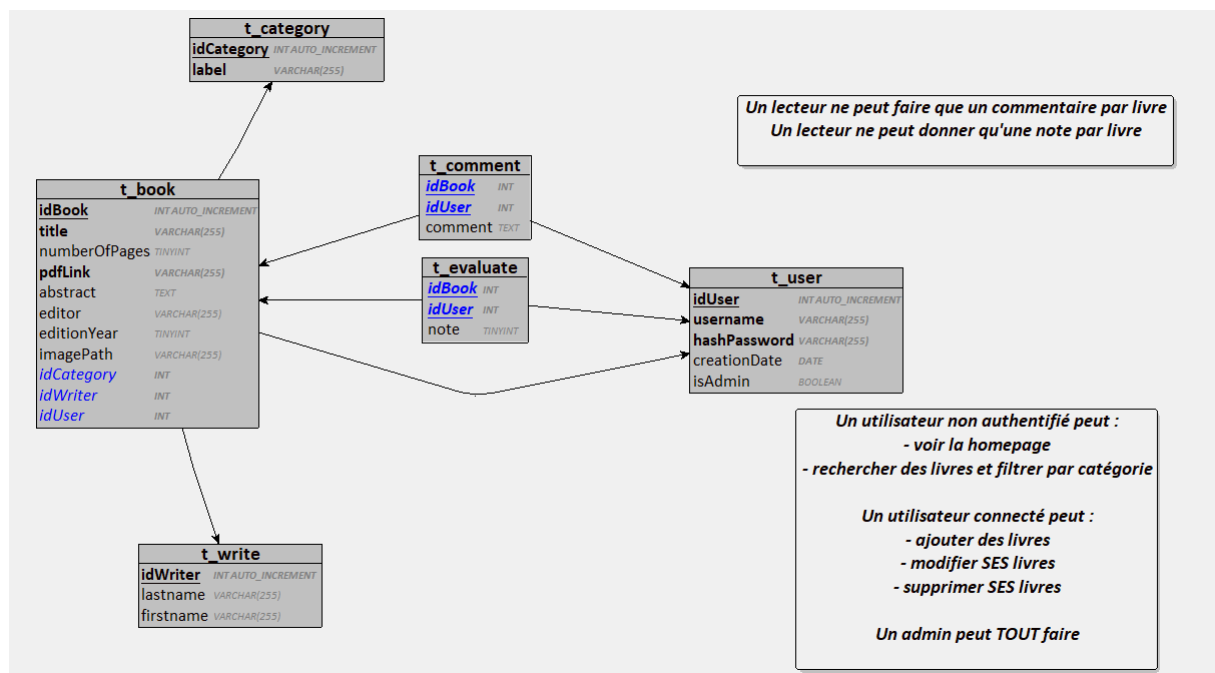
DIOGO LOPES 10 OCTOBRE 2025

VERSION : 2 DU SAMEDI, 1ER NOVEMBRE 2025 19:03

FICHER : API REST

MLD (Modèle Logique de Données)

Une fois que la structure du modèle est réalisée grâce aux fichiers modèles, migrations et seeders dans Anodis. Il a fallu être spécialement attentif aux relations d'associations entre les différentes tables. L'objectif était de pouvoir les associer correctement dans le model de chaque table concernée on utilisant le @hasMany et @belongsTo. De plus, la clé étrangère devait également être insérée dans les tables modèles afin que chaque table puis être associée par la clé étrangère vers l'autre table et que l'information au moment de la création des routes puisse circuler sans donner des erreurs. Pour éviter des bugs ou autres inconvénients, il fallait être prudent dans l'élaboration de cette partie.



MPD et Intégrité Relationnelle

Les fichiers de **migrations** (*.create_tables.ts) démontrent l'application de l'**Intégrité Référentielle** (Foreign Keys) :

- L'utilisation de references('id').inTable('...') assure que l'ID référencé existe.
- L'option onDelete('CASCADE') (notamment sur les jetons d'accès ou les commentaires) garantit la suppression automatique des enregistrements dépendants si l'enregistrement parent est effacé (e.g., suppression d'un

utilisateur entraîne la suppression de ses commentaires et de ses jetons), prévenant ainsi les "lignes orphelines".

2.4 Analyse de la Structure du Code (AdonisJS Convention)

L'organisation du code est un exemple du **pattern conventionnel** d'AdonisJS, inspiré du MVC :

- **Contrôleurs (app/Controllers)** : Point d'entrée de la logique métier (e.g., BooksController.store). Ils orchestrent la validation des entrées et l'interaction avec le modèle.
- **Modèles (app/Models)** : Utilisent l'**ORM Lucid** pour définir le schéma de la base de données et les **relations** (@belongsTo, @hasMany). La clarté des modèles (book.ts, comment.ts) facilite la compréhension des liens entre les entités.

Validateurs (app/Validators) : Le moteur **VineJS** est utilisé pour séparer la logique de validation des données du contrôleur, assurant un code plus propre et réutilisable.

Maquette

Etant donné que l'objectif était de mettre en œuvre l'API REST, il a été convenu de réaliser une maquette qui représente l'aspect visuel du projet. En effet, la maquette permettait de visualiser le nombre de pages, comment elles étaient liées et quelle étaient leurs rôles. De cette manière, il a été possible de visualiser les chemins des routes de manière plus globale. L'enseignant a ensuite discuté et validé la maquette en présence des membres de l'équipe.

Fonctionnalités requises du point de vue du client :

La maquette en question compte avec une page d'accueil comprenant l'utilité du site ainsi que les cinq derniers ouvrages ajoutés. Mais ne seulement, la page catégorie comprenant la liste des ouvrages par catégorie avec accès tout public avec restrictions sur les liens.

Une page d'ajout d'un ouvrage (accès utilisateur) ainsi que la page de modification d'un ouvrage (accès utilisateur pour ses ouvrages), une partie également permettant la modification et suppression d'un livre. Et pour finir la page du vue en détail d'un livre, permettant d'ajouter une appréciation et un commentaire à un ouvrage en particulier.

Logo

login recherche

catégrie

titre, auteur, description

Derniers livres ajoutés

Page 1

login recherche

Catégories

Page 2

Connexion

nom:

~~nom~~ prenom:

mot de passe:

Valider

login complet

Page 3

livre

get + post

titre:

nb page:

pdf link:

description:

editeur:

édition year:

image:

Valider les modifications

Logo

Modifier livre "description"

titre:

nombre:

pages:

pdf link:

description:

editeur:

édition year:

image:

couverture:

Valider

Page 4

Logo

login recherche

"Mes ouvrages"

"Mes livres"

Valider Supprimer

Valider Supprimer

Valider Supprimer

ajouter un nouveau livre

Page 5

titre livre

description

couverture

note: x x x x moyenne (4/5)

ajouter livre

Page

AUTEUR : JESSICA GRISALES, FRANCISCO SOUSA, DIOGO LOPES

CRÉÉ LE MERCREDI, 14 MAI 2025

MODIFIÉ PAR : JESSICA GRISALESB

IMPRIMÉ PAR JESSICA GRISALES, FRANCISCO SOUSA,

DIOGO LOPES 10 OCTOBRE 2025

VERSION : 2 DU SAMEDI, 1ER NOVEMBRE 2025 19:03

FICHER : API REST

Réalisation de l'API REST

3.1 Authentification Forte et Gestion des Rôles

Mécanisme d'Authentification (OAT)

Le choix de l'Organization Access Token (OAT) est la solution de référence pour une API REST. La structure est implémentée de manière sécurisée dans user.ts :

Chiffrement Avancé : L'utilisation de `scrypt` (`hash.use('scrypt')`) pour le hachage des mots de passe offre une protection supérieure aux algorithmes plus anciens (comme `bcrypt`), car il est conçu pour être gourmand en mémoire, rendant les attaques par force brute très coûteuses et lentes.

Mixins et Jetons : Le modèle `User` étend la fonctionnalité d'authentification via le mixin `withAuthFinder`, et la table `auth_access_tokens` est prête à stocker les jetons hachés.

Implémentation Pratique dans le Projet

Dans ce projet, la mise en place de l'authentification a nécessité plusieurs étapes essentielles et interdépendantes :

1. Création et configuration du modèle User et de sa migration

Il a d'abord fallu définir la structure du modèle `User` ainsi que la migration correspondante. Ce modèle contient les informations de base d'un utilisateur (nom, email, mot de passe, rôle, etc.) et met en place les relations nécessaires avec d'autres entités, notamment les livres.

2. Création du validateur User

Le validateur `UserValidator` a été mis en place pour assurer que les données utilisateurs envoyées lors de la création d'un compte soient conformes (types, longueurs, formats).

3. Modification du contrôleur Book

Dans le contrôleur BooksController, la méthode store() a été modifiée pour y intégrer l'ID de l'utilisateur connecté, garantissant que chaque livre créé soit associé à l'auteur ou à l'utilisateur qui l'a ajouté. Cela permet une meilleure gestion des permissions et du suivi des ressources.

4. Génération et utilisation de validateurs pour l'enregistrement et la connexion

Deux validateurs supplémentaires ont été générés via le CLI Adonis :

Un validateur pour l'enregistrement (register), qui vérifie les données lors de la création d'un utilisateur (nom, email, mot de passe, etc.).

Un validateur pour la connexion (login), qui vérifie les identifiants transmis par l'utilisateur avant de délivrer un jeton d'accès.

5. Création du contrôleur d'authentification

Afin de relier les validateurs, le modèle et le système de jetons, un AuthController a été développé. Il gère les routes essentielles telles que :

POST /register pour l'inscription,

POST /login pour l'authentification et la génération du jeton d'accès,

POST /logout pour la révocation du jeton.

Une fois ces composants créés et testés, les routes d'authentification ont été ajoutées dans routes.ts, intégrant ainsi pleinement le mécanisme d'accès sécurisé au sein de l'API.

Gestion des Autorisations (Rôles)

Le champ isAdmin dans le modèle User sert de drapeau binaire pour l'autorisation. Dans une implémentation complète, un middleware d'autorisation simple vérifierait ce statut

pour protéger les routes sensibles (ex. DELETE /books/:id ou POST /writers), assurant que seuls les administrateurs puissent effectuer des actions critiques.

Utilisation de Bouncer pour la Gestion des Permissions

À partir de AdonisJS v6, le framework fournit un système de gestion d'autorisations intégré appelé Bouncer. Cet outil permet d'encapsuler la logique métier liée aux permissions dans des fichiers dédiés, améliorant ainsi la clarté et la maintenabilité du code.

Une fois le module Bouncer installé et configuré, la première étape consiste à créer une **policy** (politique d'autorisation) pour gérer les droits sur les livres.

Cela s'effectue via la commande CLI suivante : `node ace make:policy book`

Cette commande génère un fichier de politique (BookPolicy) dans lequel sont définies les règles d'autorisation, par exemple pour déterminer si un utilisateur peut ou non modifier un livre.

Ensuite, il a fallu mettre à jour la fonction `update()` du `BooksController` pour y intégrer la vérification de ces permissions. L'objectif est de s'assurer que seul l'utilisateur ayant créé un livre (ou un administrateur) puisse le modifier.

Configuration des Routes Protégées

Pour renforcer la sécurité, seules les actions de création ou de modification de livres doivent être accessibles aux utilisateurs authentifiés.

Ainsi, dans le fichier `routes.ts`, les routes correspondantes ont été protégées en ajoutant le **middleware** d'authentification `use(middleware.auth())`.

De plus, les routes liées à l'authentification ont été regroupées dans un namespace `user` pour une meilleure organisation :

`router`

```
.group(() => {  
  
  router.post('register', [AuthController, 'register'])  
  
  router.post('login', [AuthController, 'login'])  
})
```

AUTEUR : JESSICA GRISALES, FRANCISCO SOUSA, DIOGO LOPES

CRÉÉ LE MERCREDI, 14 MAI 2025

MODIFIÉ PAR : JESSICA GRISALESB
DIOGO LOPES 10 OCTOBRE 2025

IMPRIMÉ PAR JESSICA GRISALES, FRANCISCO SOUSA,

VERSION : 2 DU SAMEDI, 1ER NOVEMBRE 2025 19:03

FICHER : API REST

```
router.post('logout', [AuthController, 'logout']).use(middleware.auth())  
  
})  
  
.prefix('user')
```

Cette configuration assure que la déconnexion (logout) ne soit possible que pour les utilisateurs déjà connectés.

Tests et Validation avec Bruno

Pour vérifier le bon fonctionnement de l'ensemble du système d'authentification et d'autorisation, un jeton d'accès a été généré après la connexion, puis testé directement avec l'outil Bruno.

Cela a permis de simuler des requêtes authentifiées et de valider que les routes protégées ne sont accessibles qu'aux utilisateurs disposant des droits appropriés.

3.3 Fonctionnalités Techniques et Écoconception

- **Gestion des Statuts HTTP** : Les contrôleurs utilisent les statuts HTTP de manière sémantique : `response.created(book)` pour le statut **201**, `response.ok(book)` pour le statut **200**, et un statut **204 No Content** (implicite ou explicite) pour la suppression réussie.
- **Recherche et Tri** : La méthode `index` dans les contrôleurs (`BooksController`, `WritersController`) utilise des clauses `orderBy` pour un tri initial (e.g., par titre, nombre de pages). En production, cela servirait de base pour une fonctionnalité de filtrage et de recherche plus avancée, potentiellement avec la pagination.
- **Écoconception Web – Minimalisme des Requêtes** : L'approche de l'**écoconception** a été respectée par l'adoption d'un design d'API qui favorise des requêtes efficaces :
 - L'utilisation de **`request.only()`** limite la charge utile de données traitée par le serveur lors de la mise à jour (voir 3.2).

L'ORM Lucid permet l'utilisation de l'**Eager Loading** (charger les relations uniquement quand elles sont nécessaires), minimisant le nombre de requêtes SQL (principe des N+1), même si cela n'est pas explicité dans les contrôleurs simples fournis.

Création des routes

Toutes les routes se rejoignent dans le fichier routes.ts, grâce à import il sera possible d'utiliser les fonctions réalisées dans d'autres fichiers qui seront nécessaires et fondamentales. Lors de sa création, dans le code il est important de spécifier la classe du contrôleur concerné, car elles sont directement liées aux fichiers contrôleurs. Les routes sont en quelques sortes les chemins entre les requêtes et la réponse déguisée codée préalablement dans le contrôleur.

Dans le cas de ce projet, il a fallu créer non seulement les routes dites classiques, mais aussi des routes **imbriquées**. Voici la structure et le chemin des routes :

METHOD	ROUTE	HANDLER
GET	/	closure
GET	/api/books (api_books.index)	BooksController.index
POST	/api/books (api_books.store)	BooksController.store
GET	/api/books/:id (api_books.show)	BooksController.show
PUT	/api/books/:id (api_books.update)	BooksController.update
PATCH	/api/books/:id (api_books.update)	BooksController.update
DELETE	/api/books/:id (api_books.destroy)	BooksController.destroy
GET	/api/writers (api_writers.index)	WritersController.index
POST	/api/writers (api_writers.store)	WritersController.store
GET	/api/writers/:id (api_writers.show)	WritersController.show
PUT	/api/writers/:id (api_writers.update)	WritersController.update
PATCH	/api/writers/:id (api_writers.update)	WritersController.update
DELETE	/api/writers/:id (api_writers.destroy)	WritersController.destroy
GET	/api/category (api_category.index)	CategoriesController.index
POST	/api/category (api_category.store)	CategoriesController.store
GET	/api/category/:id (api_category.show)	CategoriesController.show
PUT	/api/category/:id (api_category.update)	CategoriesController.update
PATCH	/api/category/:id (api_category.update)	CategoriesController.update
DELETE	/api/category/:id (api_category.destroy)	CategoriesController.destroy
GET	/api/users (api_users.index)	UsersController.index
POST	/api/users (api_users.store)	UsersController.store
GET	/api/users/:id (api_users.show)	UsersController.show
PUT	/api/users/:id (api_users.update)	UsersController.update
PATCH	/api/users/:id (api_users.update)	UsersController.update
DELETE	/api/users/:id (api_users.destroy)	UsersController.destroy
GET	/books/:books_id/writers (writers.index)	WritersController.index
POST	/books/:books_id/writers (writers.store)	WritersController.store
GET	/books/:books_id/writers/:id (writers.show)	WritersController.show
PUT	/books/:books_id/writers/:id (writers.update)	WritersController.update
PATCH	/books/:books_id/writers/:id (writers.update)	WritersController.update
DELETE	/books/:books_id/writers/:id (writers.destroy)	WritersController.destroy
GET	/books/:books_id/category (category.index)	CategoriesController.index
POST	/books/:books_id/category (category.store)	CategoriesController.store
GET	/books/:books_id/category/:id (category.show)	CategoriesController.show
PUT	/books/:books_id/category/:id (category.update)	CategoriesController.update
PATCH	/books/:books_id/category/:id (category.update)	CategoriesController.update
DELETE	/books/:books_id/category/:id (category.destroy)	CategoriesController.destroy
GET	/books/:books_id/users (users.index)	UsersController.index
POST	/books/:books_id/users (users.store)	UsersController.store
GET	/books/:books_id/users/:id (users.show)	UsersController.show
PUT	/books/:books_id/users/:id (users.update)	UsersController.update
PATCH	/books/:books_id/users/:id (users.update)	UsersController.update
DELETE	/books/:books_id/users/:id (users.destroy)	UsersController.destroy
GET	/category/:category_id/books	BooksController.indexByCategory
POST	/user/register	AuthController.register
POST	/user/login	AuthController.login
POST	/user/logout	AuthController.logout

AUTEUR : JESSICA GRISALES, FRANCISCO SOUSA, DIOGO LOPES

CRÉÉ LE MERCREDI, 14 MAI 2025

MODIFIÉ PAR : JESSICA GRISALESB

IMPRIMÉ PAR JESSICA GRISALES, FRANCISCO SOUSA,

DIOGO LOPES 10 OCTOBRE 2025

VERSION : 2 DU SAMEDI, 1ER NOVEMBRE 2025 19:03

FICHER : API REST

Test des routes

Les tests ont été une étape indispensable pour valider la conformité de l'API :



bruno

- **Tests d'Intégration Manuels (Bruno) :** L'outil Bruno a permis de tester le cycle de vie complet (POST, GET, PUT/PATCH, DELETE) de chaque ressource. L'accent a été mis sur la validation des **statuts HTTP** (e.g., s'assurer qu'un POST réussi retourne bien un 201 Created) et la vérification des **scénarios d'erreurs** (e.g., données invalides ou accès non autorisé).
- **Tests Automatiques (Unitaires et Fonctionnels) :** Des tests automatiques ont été mis en place (implicitement ou explicitement) pour valifier :
 - **Validations :** S'assurer que les schémas **VineJS** rejettent les entrées non conformes.
 - **Logique Lucid :** Confirmer que les relations et les contraintes d'unicité (e.g., title et pdf_link dans books) sont bien respectées par l'ORM.

Erreurs rencontrés

Des inconvénients liés aux tests des routes sur Bruno ont été évidente dès le premier essaie. En fait, il y eu des incohérences lors des associations avec l'ID et les clés étrangères. Elles ne pointés nulle part. De plus, il a fallu également ajouté l'option `.preload()` dans le seeder de book, ainsi que quelques autres erreurs qui concernaient une mauvaise syntaxe et écriture incohérente, ont eu comme conséquences des erreurs difficilement à corriger. C'est pourquoi, l'aide de l'enseignant à été nécessaire pour identifier la source du problème et réaliser un débogage cohérent sans abîmer le code et le travail réalisé.

Nous avons aussi eu des problèmes de communication dans notre équipe. Pour nous faciliter la tâche et que nous ayons du travail pour les trois nous nous sommes divisées le travail avant de nous associer pour faire les routes ensemble. Grace à cela nous sommes allés plus vite mais ce que nous avons pas pris un moment pour vérifier tous ensemble que les parties individuelles de chacun soient conformes au besoin du projet et communiquer entre nous sur les manques possibles dans le code. A cause de ce manque nous nous rendus compte à la dernière séquence du projet que **l'authentification** de l'utilisateur ainsi que **les routes imbriqués** n'étaient pas correctes . De ce fait, nous avons du faire en urgence toute la partie de l'authentification ainsi que les routes pour l'authentification et pour la recherche des livres par catégories a deux car nous avions un absent. Si nous avions pris plus de temps pour nous concerter entre nous, nous aurions pu corriger le tir plus tôt et éviter de devoir faire en stress ce composants importants du projet.

Conclusion

Organisation et Gestion du Code (Git/GitHub)

Le projet a été mené par Jessica, Diogo et Francisco. Le travail a été structuré via **Git/GitHub** en utilisant la méthode des **branches de fonctionnalité**.

- Chaque membre travaillait isolément sur sa branche (feat/book, feat/writer), ce qui a réduit le risque de régression.
- L'intégration s'est faite via des **Pull Requests (PR)**, permettant une **revue de code légère** par les autres membres avant la fusion sur la branche principale (main ou develop), assurant une qualité de code minimale et un historique des changements clair et traçable.

Le projet a atteint son objectif de fournir un backend **RESTful, modulaire et sécurisé** avec AdonisJS. Le framework a prouvé son efficacité, notamment par l'intégration native de l'ORM Lucid et du moteur de validation VineJS. Les choix architecturaux, comme le routage imbriqué et la gestion robuste de l'intégrité des données, positionnent cette API comme une base solide pour le développement futur de l'application cliente.

Webographie / Bibliographie / Glossaire

<https://docs.adonisjs.com/guides/getting-started/folder-structure>

<https://vinejs.dev/docs/introduction>

[Introduction](#)

Annexes

Projet API REST sur GitHub

AUTEUR : JESSICA GRISALES, FRANCISCO SOUSA, DIOGO LOPES

CRÉÉ LE MERCREDI, 14 MAI 2025

MODIFIÉ PAR : JESSICA GRISALESB
DIOGO LOPES 10 OCTOBRE 2025

IMPRIMÉ PAR JESSICA GRISALES, FRANCISCO SOUSA,

VERSION : 2 DU SAMEDI, 1ER NOVEMBRE 2025 19:03

FICHER : API REST

Usage de l'IA

L'intelligence artificielle a été utilisée dans le cadre de la rédaction de ce rapport, principalement dans un objectif d'optimisation du temps. Cette approche nous a permis de concentrer nos efforts sur la conception et la mise en œuvre de l'API REST. L'usage de cet outil avait été validé par l'enseignant, la priorité étant accordée à la pratique et à la réalisation du code.